MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES.

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY (SCIT).

DEPARTMENT OF COMPUTER SCIENCE.

CSC 1202: SOFTWARE DEVELOPMENT PROJECT

ILES TECHNICAL DOCUMENTATION

GROUP 1 MEMBERS.

1. AINEBYONA JOEL - 25/U/03313/EVE.
2. OLIETHO GODFREY JOSIAH - 25/U/03554/EVE
3. SSEBINA CHARLES - 25/U/03567/EVE
4. KYAMANYWA JOHN BOSCO - 25/U/03428/EVE

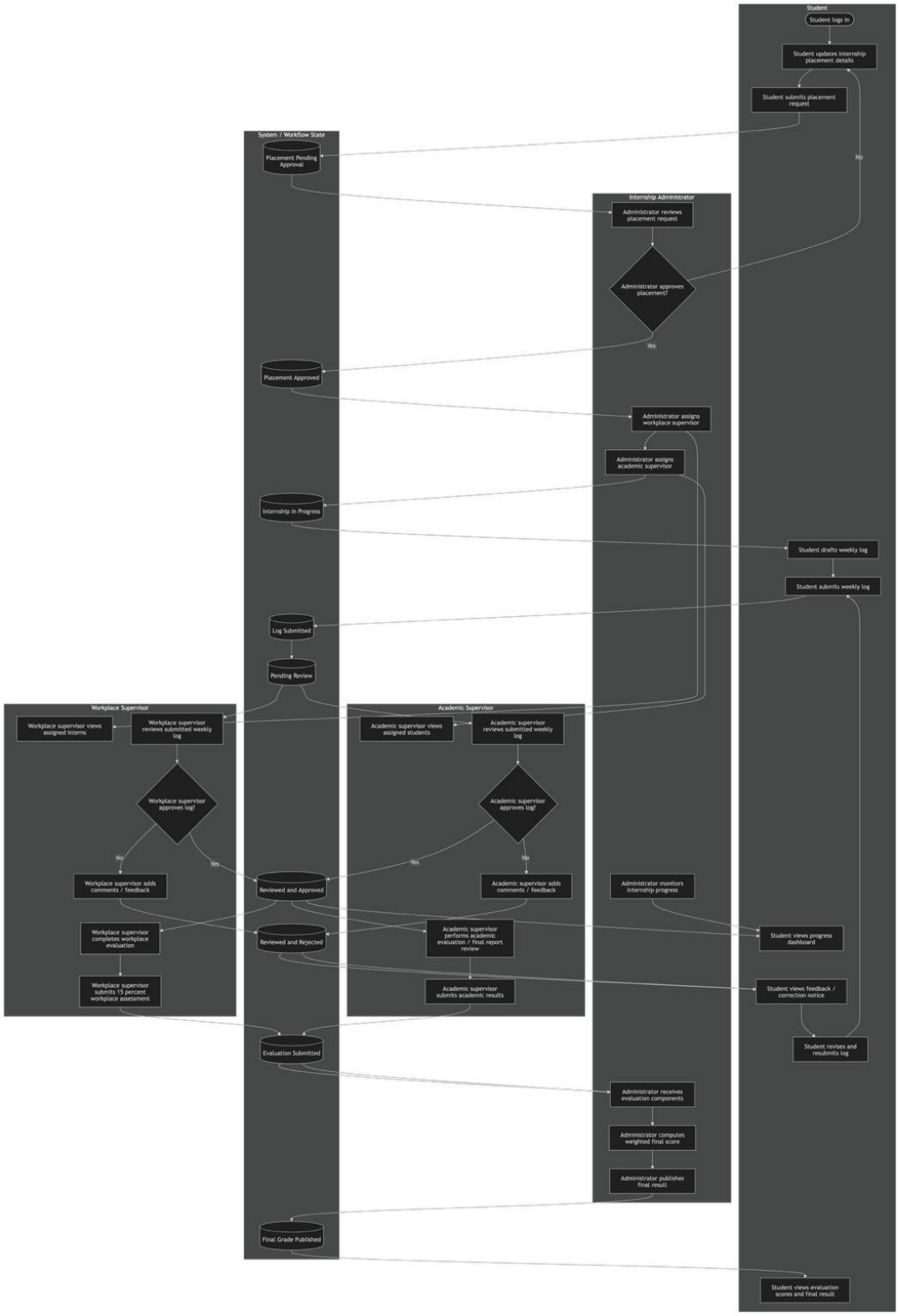


Figure 1: ILES workflow diagram reference

Table of Contents

1. Project Overview
2. System Architecture
3. Technology Stack
4. Backend Application Design
5. Frontend Application Design
6. Data Model and Relationships
7. API Endpoint Summary
8. Authentication and Authorization
9. Business Rules and Validation Logic
10. Evaluation and Results Computation
11. Testing and Coverage Plan
12. Deployment Guide
13. Maintenance and Troubleshooting
Appendix A. Workflow Diagram Reference

1. Project Overview

The Internship Logging and Evaluation System (ILES) is a full-stack web application designed to support internship placement management, weekly logbook submission, supervisor review, evaluation scoring, dashboard reporting and final result publication.

- Backend: Django with Django REST Framework.
- Frontend: React with API service modules and role-specific pages.
- Database: PostgreSQL in development, with Django ORM models and migrations.
- Users: Student Intern, Academic Supervisor, Workplace Supervisor and Internship Administrator.
- Core feature: a workflow-driven internship assessment process from placement request to final grade.

2. System Architecture

ILES follows a client-server architecture. The React frontend communicates with the Django REST API over HTTP. The backend performs authentication checks, validation, persistence and email notifications. PostgreSQL stores users, placements, weekly logs, feedback, evaluations, final results, audit logs and report records.

Layer	Responsibility	Main Components
Presentation Layer	Displays role-specific pages and forms.	React pages, dashboard components, API client, chart component
API Layer	Receives frontend requests and returns JSON responses.	Django REST Framework viewsets, routers, serializers
Business Logic Layer	Applies rules for placements, weekly logs, feedback and results.	Model clean methods, viewset perform_create methods, serializer validation
Data Layer	Stores persistent application data.	Django models, PostgreSQL tables, migrations
Notification Layer	Sends email notifications after key events.	send_iles_email utility

3. Technology Stack

Technology	Use in ILES
Python / Django	Backend web framework and application logic.
Django REST Framework	REST API endpoints, serializers and viewsets.
PostgreSQL	Relational database for project records.
React	Frontend user interface.
JavaScript API modules	Frontend communication with backend endpoints.
Jest and React Testing Library	Frontend component tests.

Django TestCase / APITestCase	Backend model and API tests.
Coverage tools	Backend and frontend test coverage reporting.

4. Backend Application Design

4.1 App Structure

File	Purpose
issues/models.py	Defines companies, placements, weekly logs, feedback, evaluations, final results, audit logs and reports.
issues/serializers.py	Converts model instances to/from JSON and performs serializer-level validation.
issues/views.py	Defines API viewsets, queryset filtering, role checks, submit actions and creation logic.
issues/urls.py	Registers API routes using DRF DefaultRouter.
issues/tests.py	Contains backend automated tests for workflow rules and API behavior.
users/models.py	Defines custom user roles and profile models.

4.2 Important Backend Viewsets

Viewset	Responsibility
CompanyViewSet	Manage company records.
InternshipPlacementViewSet	Create, list and update placement requests according to role.
SupervisorAssignmentViewSet	Assign academic and workplace supervisors to placements.
WeeklyLogViewSet	Create weekly logs, submit logs and provide evaluation summary endpoints.
FeedbackViewSet	Create feedback while automatically linking it to the logged-in supervisor.
EvaluationViewSet	Manage evaluation records and submission status.
FinalResultViewSet	Calculate and expose student final result information.

5. Frontend Application Design

The frontend is organized by role. App.js controls which page to display depending on the logged-in user role. API modules under src/api/ wrap fetch calls to the Django backend.

Frontend Area	Main Files or Pages
Student Pages	StudentDashboard, StudentPlacementPage, StudentPlacementRequestPage, StudentsWeeklyLogsPage, StudentsFeedbackPage, StudentsEvaluationsPage, StudentsResultsPage
Supervisor Pages	SupervisorDashboard, AcademicSupervisorDashboard, WorkplaceSupervisorDashboard, SupervisorWeeklyLogsPage, SupervisorFeedbackPage, SupervisorEvaluationsPage
Administrator Pages	AdminDashboard, AdminUsersPage, AdminAssessmentsPage, AdminResultsPage, SupervisorAssignmentsPage
Shared Components	ApiListPage, PerformanceTrendChart
API Modules	apiClient, weeklyLogsApi, feedbackApi, placementsApi, supervisorAssignmentsApi, finalResultsApi, usersApi

6. Data Model and Relationships

Model	Relationship Summary
Company	One company can have many internship placements.
InternshipPlacement	Belongs to a student and a company; has supervisor assignments, weekly logs, evaluations and one final result.
SupervisorAssignment	Links a supervisor to a placement as either ACADEMIC or WORKPLACE.
WeeklyLog	Belongs to a placement; has many feedback entries. Unique by placement and week number.
Feedback	Belongs to one weekly log and one supervisor. Unique by weekly log and supervisor.
Evaluation	Belongs to one placement and one evaluator. Unique by placement, evaluator and evaluation type.
EvaluationScore	Belongs to one evaluation and one criterion.
FinalResult	One-to-one with placement and calculates final mark.
AuditLog	Stores audit-trail style events.

7. API Endpoint Summary

Endpoint	Purpose
/api/issues/companies/	Company CRUD operations.

/api/issues/placements/	Placement request and placement record operations.
/api/issues/supervisor-assignments/	Supervisor assignment operations.
/api/issues/weekly-logs/	Weekly log CRUD operations.
/api/issues/weekly-logs/<id>/submit/	Submit a draft weekly log.
/api/issues/weekly-logs/my-evaluations/	Student evaluation summary for fully assessed weekly logs.
/api/issues/weekly-logs/student/<student_id>/evaluations/	Administrator view of student weekly log evaluations.
/api/issues/feedback/	Feedback creation and feedback listing.
/api/issues/evaluations/	Evaluation CRUD operations.
/api/issues/final-results/my-results/	Student final result.
/api/issues/final-results/student/<student_id>/result/	Administrator view of student final result.

8. Authentication and Authorization

Access control is enforced through role checks in viewsets and query filtering. Each role sees only the records it is allowed to access.

Role	Allowed Data Scope
Student	Own placement, own weekly logs, own feedback, own evaluations and own results.
Supervisor	Placements and weekly logs where the supervisor has an active assignment.
Administrator	All placements, users, weekly logs, feedback, assessments and results.

Security Note

The feedback API does not require supervisor_id from the frontend. The backend obtains the supervisor profile from the logged-in supervisor and saves feedback using serializer.save(supervisor=supervisor_profile).

9. Business Rules and Validation Logic

Rule	Implementation Area
A student cannot submit a second placement unless the previous placement is rejected.	InternshipPlacement.clean and InternshipPlacementViewSet.perform_create

A weekly log is unique per placement and week number.	WeeklyLog unique constraint and WeeklyLogSerializer.validate
Submitted weekly logs cannot be submitted again.	WeeklyLogViewSet.submit
Only a student can submit their own weekly logs.	WeeklyLogViewSet.submit permission checks
Only an assigned supervisor can give feedback.	Feedback.clean and FeedbackViewSet.perform_create
One supervisor can give only one feedback entry per weekly log.	Feedback unique constraint and FeedbackViewSet.perform_create
Supervisor type must match assignment role.	SupervisorAssignment.clean
Final weekly log mark requires both academic and workplace scores.	WeeklyLog.get_average_supervisor_score

10. Evaluation and Results Computation

Each weekly log can receive one academic supervisor score and one workplace supervisor score. The weekly log final mark is calculated only after both scores exist.

Computation	Formula / Meaning
Academic Score	Latest score from an academic supervisor feedback entry.
Workplace Score	Latest score from a workplace supervisor feedback entry.
Final Weekly Log Mark	(Academic Score + Workplace Score) / 2
Average Weekly Logs Score	Average of all fully assessed weekly log final marks.
Final Mark	Weekly logs score + supervisor evaluation score + final report score + workplace assessment score, capped at 100.

11. Testing and Coverage Plan

The project test plan includes ten backend tests and five frontend tests. These tests focus on the main workflow and the recent system fixes.

Backend Test Area	Examples
Placement	Create placement request; block duplicate active placement.
Weekly Logs	Create weekly log; block duplicate weekly log; submit draft; block resubmission.
Feedback	Assigned supervisor submits feedback; duplicate feedback blocked; unassigned supervisor blocked.
Results	Weekly log average and final result calculation.

Frontend Test Area	Examples
Student Placement Page	Shows active placement warning and blocks second request.
Placement Form	Submits placement details when allowed.
Supervisor Weekly Logs Page	Shows feedback form before review and hides it after review.
Feedback Payload	Submits feedback without supervisor_id.

Backend coverage can be generated with coverage run manage.py test issues and coverage report -m. Frontend coverage can be generated using npm test-----coverage --watchAll=false.

12. Deployment Guide

12.1 Backend Deployment Checklist

- Set DEBUG=False in production.
- Configure ALLOWED_HOSTS for the deployed domain.
- Configure PostgreSQL credentials securely using environment variables.
- Run python3 manage.py migrate on the production database.
- Collect static files if Django serves static assets.
- Configure email backend for notification emails.
- Use a production WSGI/ASGI server and reverse proxy.

12.2 Frontend Deployment Checklist

- Update API_BASE_URL in apiClient.js to point to the deployed backend API.
- Run npm install to install dependencies.
- Run npm run build to create the production frontend build.
- Deploy the build folder to the selected hosting environment.
- Ensure CORS and CSRF settings allow the frontend domain.

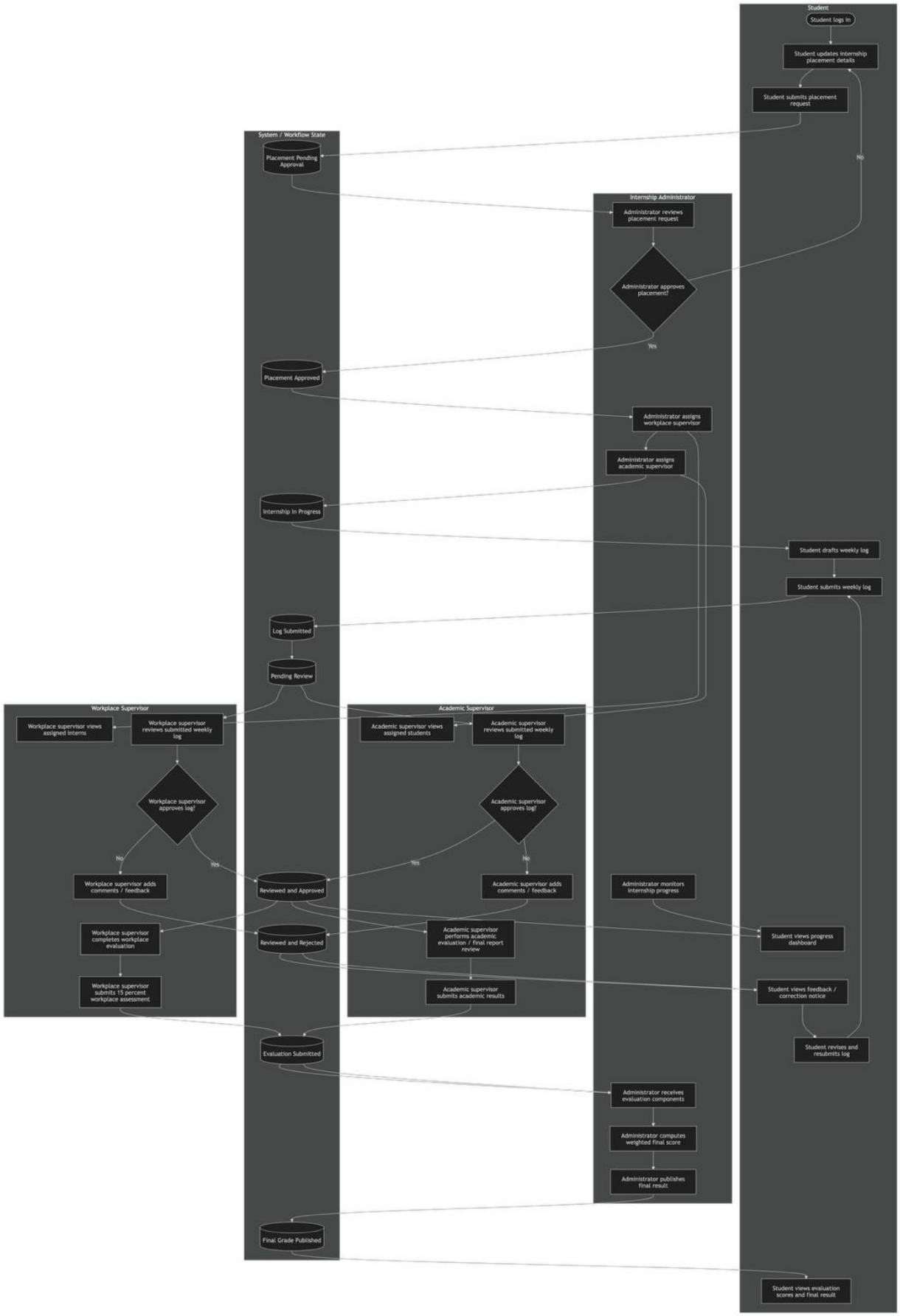
13. Maintenance and Troubleshooting

Issue	Likely Cause	Resolution
Connection refused to PostgreSQL	PostgreSQL server is not running.	Start PostgreSQL service and confirm port 5432.
API error 400	Validation failed.	Read frontend error message from apiClient and correct the form data.
Supervisor Id required	Old serializer still contains supervisor_id.	Remove supervisor_id from FeedbackSerializer and restart Django.
Duplicate feedback migration fails	Existing duplicate feedback records in database.	Delete duplicate feedback records then rerun migration.
Frontend cannot connect to backend	Django server is not running or API URL is wrong.	Start backend server and confirm API_BASE_URL.
Student evaluations not appearing	Both supervisors have not scored the	Wait until both supervisor scores exist.

	log.	

Appendix A. Workflow Diagram Reference

The following workflow diagram shows the operational flow between student, internship administrator, academic supervisor, workplace supervisor and system workflow states.


Student
Student logs in
Student updates internship placement details
Student submits placement request
No
System / Workflow State
Placement Pending Approval
Internship Administrator
Administrator reviews placement request
Administrator approves placement?
Yes
Placement Approved
Administrator assigns workplace supervisor
Administrator assigns academic supervisor
Internship In Progress
Student drafts weekly log
Student submits weekly log
Log Submitted
Pending Review
Workplace Supervisor
Workplace supervisor views assigned interns
Workplace supervisor reviews submitted weekly log
Workplace supervisor approves log?
No
Yes
Workplace supervisor adds comments / feedback
Workplace supervisor completes workplace evaluation
Workplace supervisor submits 15 percent workplace assessment
Academic Supervisor
Academic supervisor views assigned students
Academic supervisor reviews submitted weekly log
Academic supervisor approves log?
Yes
No
Academic supervisor adds comments / feedback
Academic supervisor performs academic evaluation / final report review
Academic supervisor submits academic results
Reviewed and Approved
Reviewed and Rejected
Administrator monitors internship progress
Student views progress dashboard
Student views feedback / correction notice
Student revises and resubmits log
Evaluation Submitted
Administrator receives evaluation components
Administrator computes weighted final score
Administrator publishes final result
Final Grade Published
Student views evaluation scores and final result